

Reviewer Pack — Minimal Rank of Geometric Langlands Duality Bounds Frege Pro...

Entry d8d33cab4fb6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 20:15:15 UTC

Minimal Rank of Geometric Langlands Duality Bounds Frege Proof Length

Entry ID: d8d33cab4fb6 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 20:15:15 UTC

1. Conjecture

Field A (mathematical branch): Geometric Langlands Program **Field B** (complexity object): Complexity Theory: Frege Proof Complexity

Statement:

{‘sentence_1’: ‘For a given Boolean function f , the minimal rank of its associated geometric Langlands dual object is a lower bound on the length of any Frege proof for f .’, ‘sentence_2’: ‘Formally, if $L(f)$ is the geometric Langlands dual object of f , then the minimal rank $r(L(f))$ satisfies: $\forall f \in \{0,1\}^n$, there exists a Frege proof of f with length $\leq 2^{\{r(L(f))\}}$.’, ‘sentence_3’: ‘This holds for all $n \leq 40$ and 30 random seeds.’}

Rationale (proposer’s reasoning):

{‘sentence_1’: ‘The Geometric Langlands Program provides a bridge between algebraic geometry and number theory, which may offer new perspectives on complexity theory.’, ‘sentence_2’: ‘Geometric Langlands duality maps geometric objects to other geometric objects, which could potentially lead to lower bounds in proof complexity by exploiting geometric properties.’, ‘sentence_3’: ‘Frege proofs are a central object of study in propositional proof complexity, and a connection with geometric Langlands would be novel.’}

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 218e7fdfa3fe5b53

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

If the minimal rank of the geometric Langlands dual object of a Boolean function f satisfies $r(L(f)) \leq 2^{\{\text{length_of_Frege_proof}\}}$ for all seeds and $n \leq 40$, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Geometric Langlands Program" AND "Frege proof complexity" - "minimal rank of geometric Langlands dual object" AND "Frege proof length" - "lower bound on Frege proof for Boolean function" AND "geometric Langlands duality"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.6s

5.1 Generated Python source

```
import random
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def construct_geometric_langlands_dual(f):
        # Placeholder function to simulate the construction of a geometric Langlands dual object
        # This is a dummy implementation and does not reflect actual Geometric Langlands Duality
        return [sum(f[i:i+2]) % 2 for i in range(len(f) - 1)]

    def minimal_rank(matrix):
        n = len(matrix)
        rank = 0
        for col in range(n):
            if any(matrix[row][col] != 0 for row in range(rank, n)):
                rank += 1
                for row in range(rank, n):
                    factor = matrix[row][col] / matrix[rank-1][col]
                    for j in range(col, n):
                        matrix[row][j] -= factor * matrix[rank-1][j]
        return rank

    def frege_proof_length(f):
        # Placeholder function to simulate the length of a Frege proof
        # This is a dummy implementation and does not reflect actual Frege proofs
        return len(f)

    n = random.choice([5, 10, 15, 20, 30, 40])
    f = generate_boolean_function(n)
```

```

L_f = construct_geometric_langlands_dual(f)
r_L_f = minimal_rank(L_f)
length_of_Frege_proof = frege_proof_length(f)

conjecture_holds = r_L_f <= 2**length_of_Frege_proof
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "minimal_rank",
    "metric_value": r_L_f,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, **{trial_result}}}")
        results.append(trial_result)

    metric_values = [r["metric_value"] for r in results]
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={sum(metric_values)/len(metric_values)} std=0.0 support_fr")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b73c8b8e.py", line 69, in <module>
    trial_result = run_trial(seed)
    ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b73c8b8e.py", line 49, in run_trial
    r_L_f = minimal_rank(L_f)
    ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b73c8b8e.py", line 33, in minimal_rank
    if any(matrix[row][col] != 0 for row in range(rank, n)):
    ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b73c8b8e.py", line 33, in <genexpr>
    if any(matrix[row][col] != 0 for row in range(rank, n)):

```

~~~~~^  
TypeError: 'int' object is not subscriptable

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test code crashed during execution, which prevents us from verifying the conjecture's support conditions. | next: Investigate and fix the error in the test code to proceed with the verification of the conjecture.

## 11. Audit log (LLM calls)

**Total LLM calls:** 9

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | glm4:latest      | 0         | 0   | 10579        |
| 2 | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 6164         |
| 3 | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 4758         |
| 4 | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 5326         |
| 5 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 11424        |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 11565        |
| 7 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 9479         |
| 8 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 8532         |
| 9 | judge           | ollama_remote | glm4:latest      | 0         | 0   | 12318        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 80146 ms total latency. Provider mix: {'ollama\_remote': 9}

(full prompt+response transcripts available in `research/audit/d8d33cab4fb6.jsonl`)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d8d33cab4fb6.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/d8d33cab4fb6.tar.gz` (if generated)